Experiment 1: String Constructors

Problem Statement:
Write a Java program to demonstrate different ways of creating and initializing `String` objects using various **String constructors**.

Requirements:

- Create a String using a string literal.
- Create a String using `new String()`.
- Create a String from a character array.
- Create a String from a byte array.
- Display all the created strings.
- Demonstrate the difference between String objects created using literals and `new`.

OUTPUT:-

```
C:\Users\ITLAB2\Desktop>javac StringConstructors.java

C:\Users\ITLAB2\Desktop>java StringConstructors
String using literal: Hello Java
String using new String(): Welcome to Java
String from character array: Java
String from byte array: Hello

String Comparison:
s5 == s6 : true
s5 == s7 : false
s5.equals(s7) : true
```

Experiment 2: StringBuffer Class

Problem Statement:
Write a Java program to demonstrate the functionality of the **StringBuffer** class by performing various mutable string operations.

Requirements:

- Create a `StringBuffer` object.
- Append text to the buffer.
- Insert text at a specified position.
- Replace a portion of the string.
- Delete characters from the buffer.
- Reverse the contents.
- Display the current length and capacity of the `StringBuffer`.

OUTPUT:-

```
C:\Users\ITLAB2\Desktop>javac StringBufferDemo.java

C:\Users\ITLAB2\Desktop>java StringBufferDemo
Initial String: Hello
After append: Hello Java
After insert: Hello World Java
After replace: Hello AmazingJava
After delete: Hello ava
After reverse: ava olleH
Length: 9
Capacity: 21
```

Experiment 3: StringTokenizer Class

Problem Statement:
Write a Java program to tokenize a given sentence using the **StringTokenizer** class and display each individual token.

Requirements:

- Read a sentence from the user.
- Use `StringTokenizer` to split the sentence into words.
- Display each token separately.
- Display the total number of tokens.
- Repeat the operation using a user-specified delimiter.

OUTPUT:-

```
:\Users\ITLAB2\Desktop>javac StringTokenizerDemo.java

:\Users\ITLAB2\Desktop>java StringTokenizerDemo
nter a sentence: Charan

okens using space:
haran
otal number of tokens: 1

nter a delimiter: r
okens using delimiter:
ha
an
otal number of tokens: 0
```

Inheritance

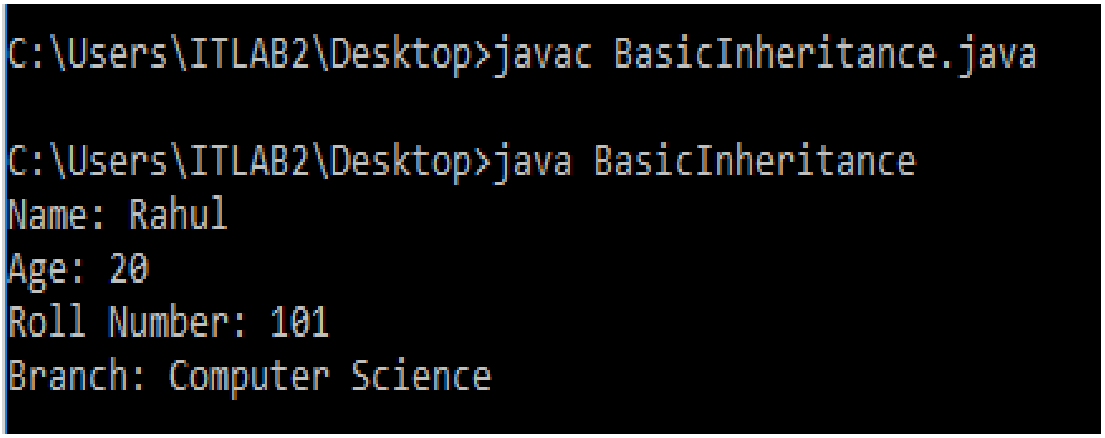
Experiment 4: Basic Inheritance

Problem Statement:
Write a Java program to demonstrate **single inheritance** by creating a superclass containing common properties and methods and a subclass that inherits and extends the functionality of the superclass.

Requirements:

- Create a superclass `Person` with attributes such as name and age.
- Define a method to display the person details.
- Create a subclass `Student` that inherits from `Person`.
- Add student-specific attributes such as roll number and branch.
- Display all details using the subclass object.

OUTPUT:-



Experiment 5: Using `super` Keyword

Problem Statement:
Write a Java program to demonstrate the use of the `super` **keyword** for accessing superclass variables, methods, and constructors.

Requirements:

- Create a superclass `Employee` with appropriate variables and methods.
- Create a subclass `Manager`.
- Use `super` to access a superclass variable.
- Use `super` to invoke a superclass method.
- Use `super()` to invoke the superclass constructor.

- Display the complete employee and manager details.

OUTPUT:-

